# Série 10

## Objectifs

Comprendre le concept de **liste**.

Mettre en œuvre le concept de **boucle** pour résoudre des problèmes simples et traiter des **listes**.

## Notions pour la série

Démarche de résolution de problèmes

## Contraintes impératives

Les différents programmes que vous devez réaliser seront nommés **ALP-S10Ex1** à **ALP-S10Ex3** et se trouveront dans un dossier nommé **ALP-S10**.

## Énoncé

### Exercice 1

Un magasin vous mandate pour réaliser un outil de contrôle de stock.
Vous devez réaliser sa première version en respectant les spécifications fournies en commentaires.

Une liste **stock** permet à l'utilisateur d'indiquer les numéros d'article qu'il a dans son stock.
Une liste **catalogue** contenant tous les numéros d'article existants.

Vous devez coder la procédure **controler_le_stock** du script Fourni_ALP-S10Ex1 qui vérifie le contenu de la liste fourni par l'utilisateur et qui affiche un message circonstancié (à l'aide de la procédure afficher_msg déjà faite) selon différents critères.

**def controler_le_stock (stock, catalogue)**
Contrôle le contenu de la liste stock et affiche un message (appel de la procédure afficher_msg) avec un code correspondant à cette analyse :

- pour chaque article du stock inconnu dans le catalogue :
  Appel afficher_msg avec code_erreur = **INCONNU**, valeur = n° article, position = indice dans stock
- pour chaque article apparaissant plusieurs fois dans le stock :
  Appel afficher_msg avec code_erreur = **MULTIPLE**, valeur = n° article, position = indice dans stock
- à la fin du contrôle, si aucune erreur n'a été détectée :
  Appel afficher_msg avec code_erreur = **OK**, valeur = nombre d'articles dans le stock
- par contre, si une ou plusieurs erreurs ont été détectées :
  Appel afficher_msg avec code_erreur = **TOTAL**, valeur = nombre d'erreurs détectées.

#### Résultats

Voici des exemples de résultats produits par la procédure de test main. L'obtention de ces résultats ne garantit évidemment pas que le programme que vous avez conçu est correct et complet. Vous devez également mener des tests d'exécution personnels, adaptés au code que vous avez développé.

Exemple avec les données 33 44 55 888 777 666 :

```
Aucune erreur. Il y a 6 articles dans le stock.
```

Exemple avec les données 33 44 99 666 555 44 :

```
Cellule d'indice 1, l'article n°44 est répertorié plusieurs fois dans le stock !
Cellule d'indice 2, l'article n°99 est inconnu dans le catalogue !
Cellule d'indice 5, l'article n°44 est répertorié plusieurs fois dans le stock !
Il y a 3 erreurs dans le stock.
```

# Exercice 2

Compléter le script Fourni_ALP-S10Ex2 de sorte que les différentes procédures-fonctions respectent les spécifications données dans leurs commentaires. Vous remplacerez les commentaires ''' A COMPLETER ''' par les éléments nécessaires.

# Exercice 3

Suite à la Course de l'Escalade qui a eu lieu ce weekend, on vous demande de faire un petit programme qui valide les classements, qui permet d'afficher certains résultats, ainsi que de modifier le temps de certains coureurs selon leur sexe.

Les résultats ont été stockés dans 2 listes lors du passage de la ligne d'arrivée :

- une liste **lst_temps** contenant le temps de chaque coureur qui a franchi la ligne d'arrivée.
  *Les temps sont donc stockés dans l'ordre d'arrivée (du premier au dernier).*
- une liste **lst_genre** indiquant si le coureur est un homme (valeur True) ou une femme.

Un même coureur est désigné au moyen du même indice dans les deux listes.

La procédure main contient les résultats de plusieurs courses par catégorie d'âge. Vous pouvez donc tester les résultats des courses n°1 à 4

Votre travail consiste à compléter le script Fourni_ALP-S10Ex3 de sorte que les différentes procédures-fonctions respectent les spécifications données dans leurs commentaires. Vous remplacerez les commentaires ''' A COMPLETER ''' par les éléments nécessaires.

## Résultats

Voici les résultats que vous devez obtenir pour un certain nombre de valeurs test. L'obtention de ces résultats ne garantit évidemment pas que le programme que vous avez conçu est correct et complet. Vous devez également mener des tests d'exécution personnels, adaptés au code que vous avez développé.

Résultats de la course n°1 :

```
Liste des hommes arrivés avant la 1ère femme :
Coureur n°0 : 12.5
Coureur n°1 : 12.7
Coureur n°2 : 12.8
3 hommes plus rapides que la 1ère femme qui a mis 13.2
```

Résultats de la course n°2 :

```
Liste des hommes arrivés avant la 1ère femme :
0 hommes plus rapides que la 1ère femme qui a mis 22.5
```

Résultats de la course n°3 :

```
Liste des hommes arrivés avant la 1ère femme :
Coureur n°0 : 32.5
Coureur n°1 : 32.7
Coureur n°2 : 32.8
Coureur n°3 : 33.2
Coureur n°4 : 33.7
Coureur n°5 : 35.1
Coureur n°6 : 35.8
Coureur n°7 : 35.8
Coureur n°8 : 36.2
Coureur n°9 : 39.7
Tous les hommes sont affichés car il n'y a pas de femme dans cette course !
```

Résultats de la course n°4 :

```
- temps du coureur n°2 erroné : 40.8 alors que le précédent a mis 42.7
- temps du coureur n°6 erroné : 45.1 alors que le précédent a mis 45.2
```

Les résultats « APRÈS MODIFICATION » ont volontairement été omis ici, afin de ne pas surcharger l'énoncé. Cependant, vous pouvez les vérifier à l'aide de la course n°3 en contrôlant que le temps de chaque coureur ait bien été incrémenté de 1.